How Do Language Models See Text? A Primer on Tokenization

October 2026

Danu Caus

Many LLM Quirks Start with Tokenization

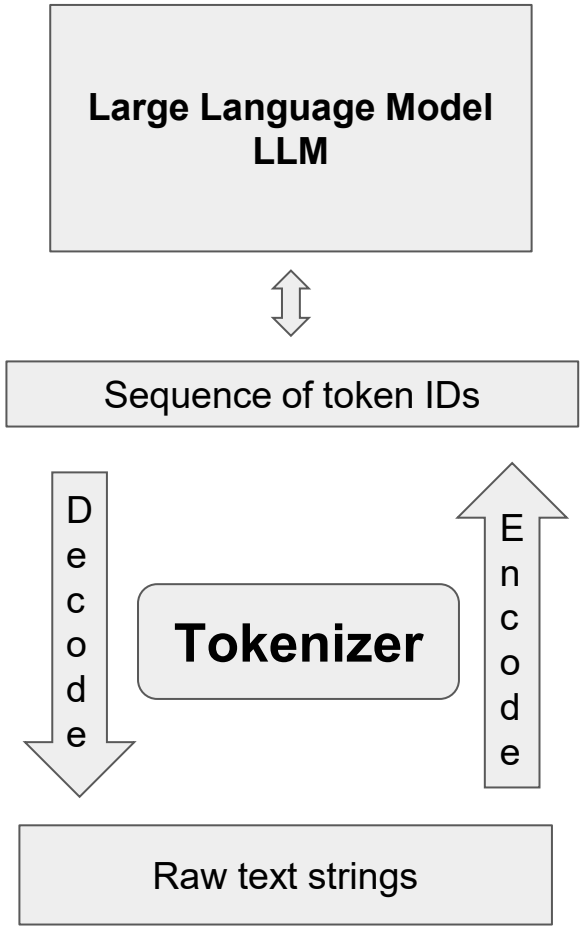
- **Letter counting:** a token can contain several letters, making character-level tasks less direct. For example, GPT-4 has incorrectly reported **two “r”s in “strawberry” instead of three**.

- **Arithmetic:** a number such as 12345 may split into 123 and 45, rather than individual digits.

- **Languages:** equivalent messages can require **different numbers of tokens**, affecting context usage.

- **Programming:** whitespace tokenization affects **how much code fits** into the context window.

Where the Tokenizer Fits



The Large Language Model (LLM) has its own training dataset, which is much larger than the dataset typically used to train the tokenizer.

The tokenizer will have its own training set of documents which are used to train the vocabulary using the Byte Pair Encoding (BPE) algorithm as a one-time procedure. Once we have the **vocabulary** and the **merges**, we can do both encoding and decoding.

A Brief History of Byte Pair Encoding

- **Byte Pair Encoding (BPE) originated as a compression algorithm.** It repeatedly replaces the most frequent pair of adjacent bytes with a single unused byte, producing a more compact representation (Gage, 1994).
- **Sennrich et al. adapted BPE for subword tokenization.** Their version merges characters and character sequences into reusable pieces, helping neural machine translation models represent **rare and unseen words** (Sennrich, Haddow & Birch, 2016).
- **GPT-2 used byte-level BPE for language modeling.** Starting from **256 possible byte values**, its tokenizer learns larger pieces and can encode any UTF-8 text without an unknown-character token (Radford et al., 2019).

How Does BPE Create New Tokens?

Example training text: hug hug hug pug pug

Step	“hug” × 3	“pug” × 2	New token added	ID
Start	[h] [u] [g]	[p] [u] [g]	None	[h] - 1, [u] - 2, [g] - 3, [p] - 4
Merge u + g (5 occurrences)	[h] [ug]	[p] [ug]	[ug]	5
Merge h + ug (3 occurrences)	[hug]	[p] [ug]	[hug]	6
Merge p + ug (2 occurrences)	[hug]	[pug]	[pug]	7

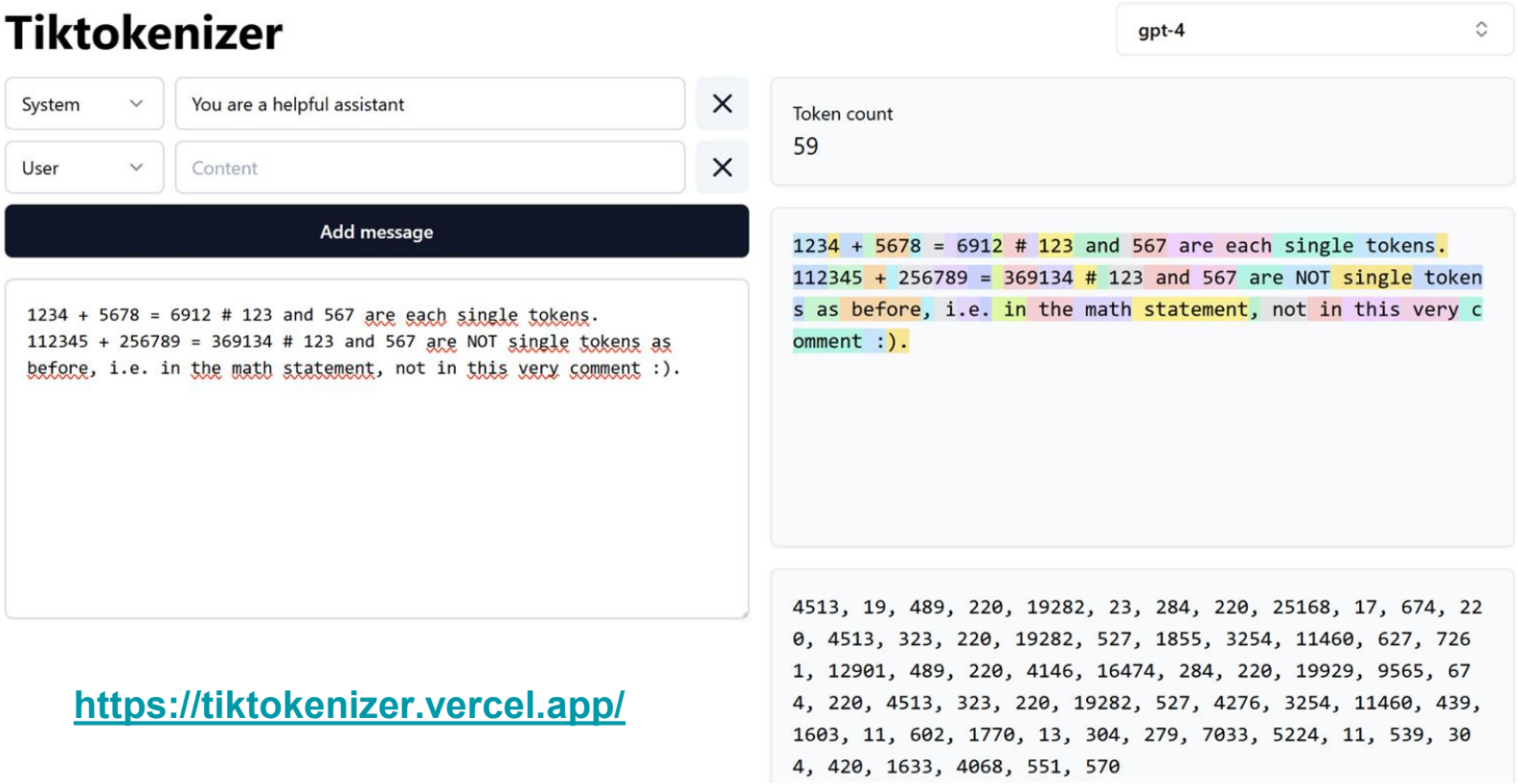
Each bracketed group is one token. We repeatedly merge the most frequent pair of neighboring tokens into a new token. In this example, merges stay within words and never cross spaces.

https://huggingface.co/learn/llm-course/en/chapter6/5

How Training Data Shapes Tokenization

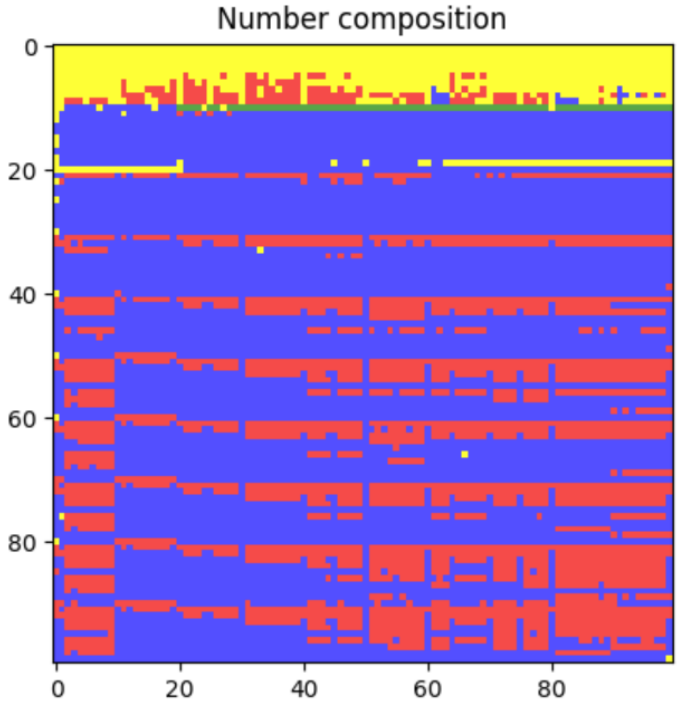
- **Different training data.** The tokenizer and LLM can use different datasets, with tokenizer training typically requiring much less text.
- **Frequent patterns get compact representations.** The training mix shapes which pieces enter the vocabulary.
- **Language balance matters.** English-heavy data can favour compact English representations over other languages.
- **Code and formatting matter too.** The training mix influences whitespace encoding and can favour formats such as YAML over JSON, or vice versa.
- **Vocabulary choices have lasting effects.** The vocabulary usually stays fixed during LLM training. Fewer tokens leave **more room in the context window**.

Numbers Split into Tokens of Variable Length



https://tiktokenizer.vercel.app/

Number Tokenization in GPT-2 / GPT-3



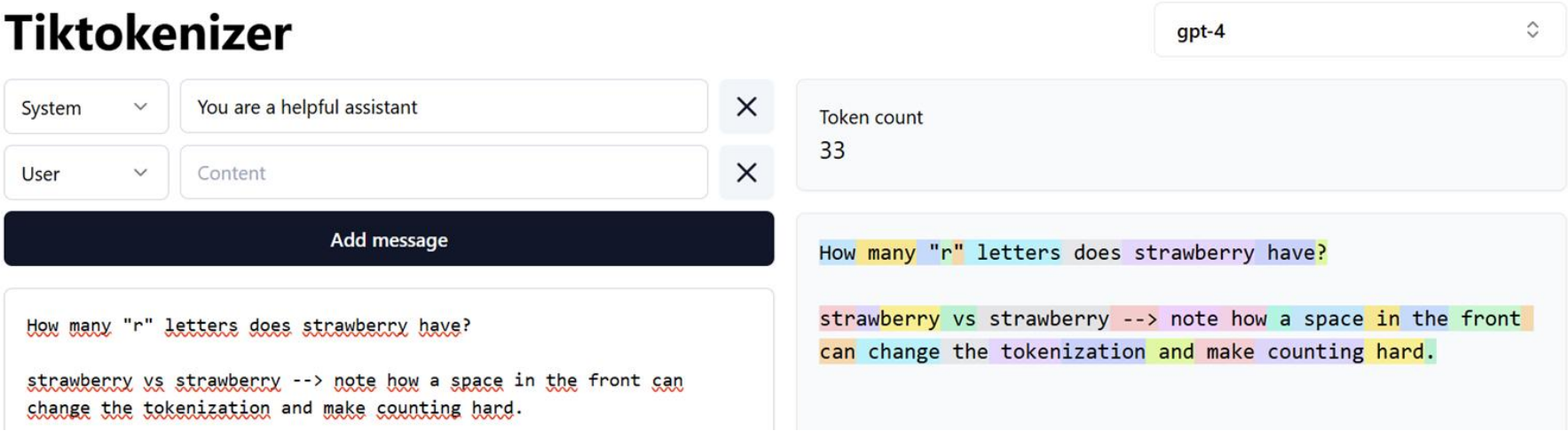


X-axis: position within a row of 100 consecutive integers.
Y-axis: which block of 100 integers you are looking at. It increases **downwards**.
Moving **one cell right** advances by **1**.
Moving **one row down** advances by **100**.

Colour	Meaning	Example
Yellow: **Unique**	The entire number is one token	`[number]`
Red: **1–3**	One digit, then three digits	`2249` → `[2][249]`
Green: **2–2**	Two digits, then two digits	`2250` → `[22][50]`
Blue: **3–1**	Three digits, then one digit	`2251` → `[225][1]`

Check the full blog post at → https://www.beren.io/2023-02-04-Integer-tokenization-is-insane/

LLM Dyslexia



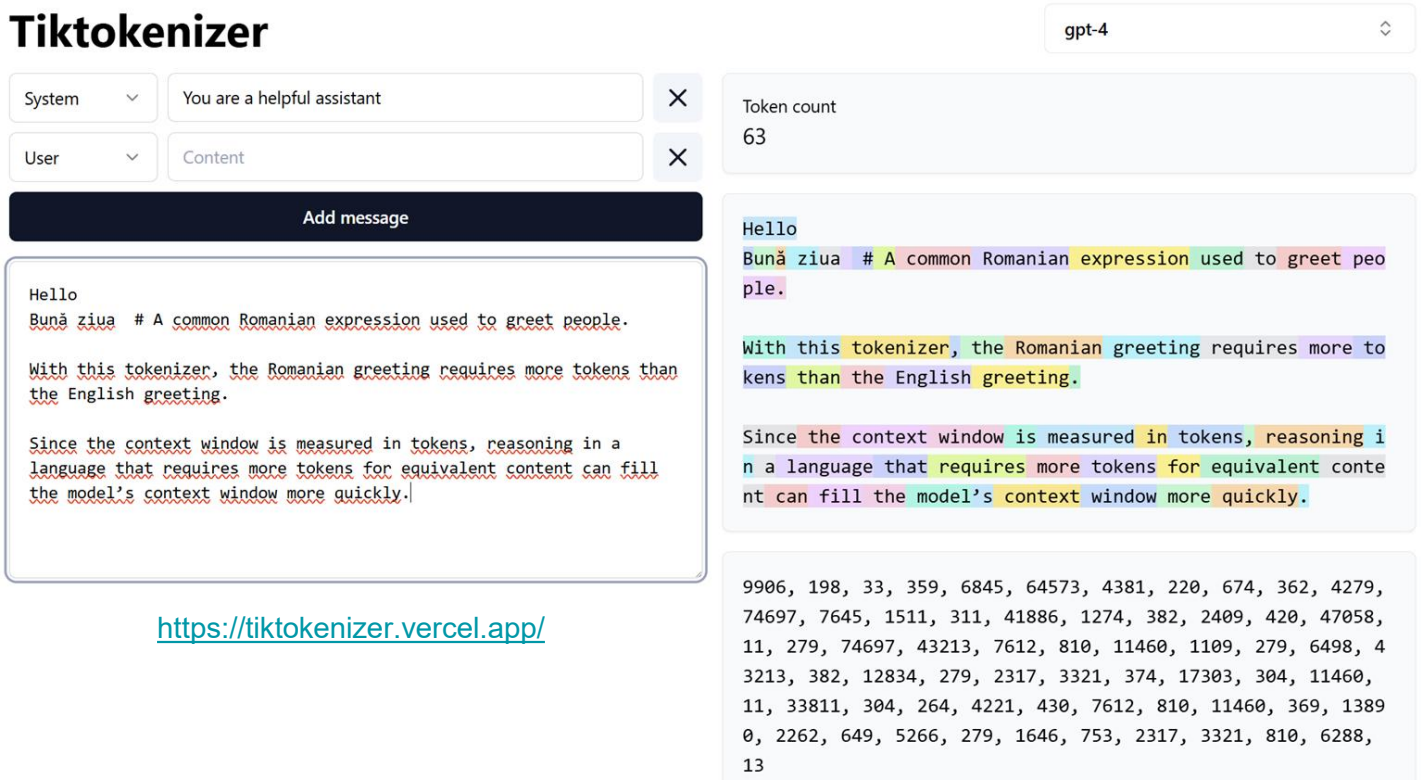
[str][aw][berry] → 3 tokens
[strawberry] → 1 token

} Same word, different IDs: the model sees token IDs, not letters.

For such a question, an agentic LLM can write and run a short piece of code that splits the word into letters and counts them.

It's important to develop an intuition about when to ask the LLM to do things via code and when to trust its outputs without additional tools.

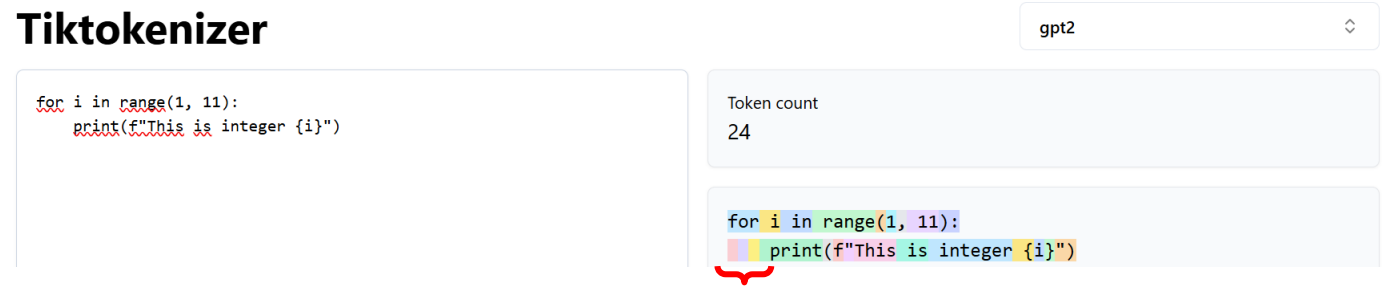
Not All Languages Are Tokenized Equally



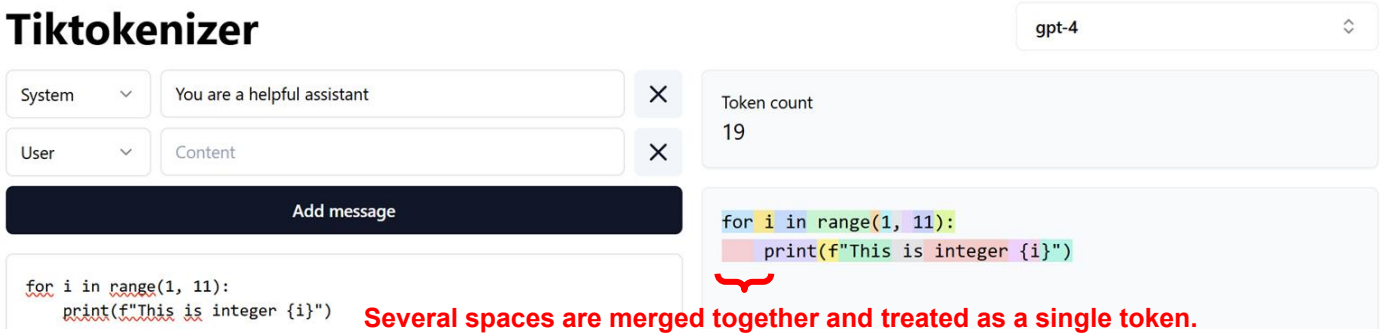
https://tiktokenizer.vercel.app/

English dominates online text, so both the LLM and its tokenizer see more of it, and English can end up being tokenized more compactly.

Tokenizing Source Code



Each space is encoded separately, resulting in longer token sequences



Several spaces are merged together and treated as a single token.

In this example, the GPT-4 tokenizer groups consecutive spaces into **fewer tokens** than GPT-2's, allowing **more code to fit within the same token budget**.

UTF-8 Encoding

Code point ↔ UTF-8 conversion

First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+007F	0yyyzzzz			
U+0080	U+07FF	110xxxyy	10yyzzzz		
U+0800	U+FFFF	1110wwww	10xxxxyy	10yyzzzz	
U+010000	U+10FFFF	11110uvv	10vvwwww	10xxxxyy	10yyzzzz

[https://en.wikipedia.org/wiki/UTF-8]

There are other encoding schemes like **UTF-16** and **UTF-32**. UTF-32, for example, is **fixed length**, whereas **UTF-8** is **variable-length**, which can result in shorter byte sequences. One of the major advantages of UTF-8 is that it is backwards compatible with **ASCII**, which is a much simpler encoding of text.

Check also → https://www.reedbeta.com/blog/programmers-intro-to-unicode/

Not All Byte Sequences Are Valid UTF-8

Example:

The character é (U+00E9) is encoded as two bytes: 11000011 10101001 → decimal values 195, 169

- ➢ 110 indicates the **start** of a two-byte sequence.
- ➢ 10 identifies the **continuation** byte.

By contrast, the byte 128 (10000000 in binary) cannot be decoded on its own: its 10 prefix marks it as a **continuation** byte, so a preceding **start** byte is required.

Hence, when you see lots of Unicode Replacement Characters like , the LLM post-processing pipeline cannot decode the bytes correctly for the browser to render. When decoding with errors="replace", invalid UTF-8 sequences are replaced with

```
def decode(token_ids):
    # Join the bytes first: a UTF-8 character may span several tokens.
    tokens = b"".join(vocab[token_id] for token_id in token_ids)
    text = tokens.decode("utf-8", errors="replace") # In case of decoding issues use Unicode Replacement Characters
    return text
```

https://github.com/ducspe/UnderstandingTokenization/blob/main/BPE_Tokenizer_Notebook.ipynb

Preventing Merges with Regex Pre-tokenization

Some merges should never take place. For example, we might want to keep a word separate from the punctuation after it. Regular expressions let us specify which parts of the text should stay separate.

gpt-2 / src / encoder.py

Code | Blame | 117 lines (104 loc) · 4.14 KB | Raw

```
42  class Encoder:
43
52          # Should haved added re.IGNORECASE so BPE merges can happen for capitalized versions of contractions
53          self.pat = re.compile(r"""'s|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+""")
54
55      def bpe(self, token):
56          if token in self.cache:
57              return self.cache[token]
58          word = tuple(token)
59          pairs = get_pairs(word)
60
61          if not pairs:
62              return token
63
```

The regex **splits text into smaller chunks before BPE is applied**. It recognizes:

- **English contractions:** 's, 't, 're, 've, 'm, 'll, 'd.
- **Groups of letters:** such as hello.
- **Groups of numbers:** such as 123.
- **Punctuation and symbols:** such as !!!.
- **Whitespace:** spaces, tabs and line breaks.

For example, **"I'm buying 12 apples!"** becomes: ["I", "'m", " buying", " 12", " apples", "!"]

BPE then processes each chunk separately and cannot merge across these boundaries.

https://github.com/openai/**gpt-2**/blob/master/src/encoder.py

GPT-2 vs GPT-4 Pre-tokenization

tiktoken / tiktoken_ext / openai_public.py ↑ Top

Code Blame 162 lines (144 loc) · 5.48 KB Raw

```
def cl100k_base():
    mergeable_ranks = load_tiktoken_bpe(
        "https://openaipublic.blob.core.windows.net/encodings/cl100k_base.tiktoken",
        expected_hash="223921b76ee99bde995b7ff738513eef100fb51d18c93597a113bcffe865b2a7",
    )
    special_tokens = {
        ENDOFTEXT: 100257,
        FIM_PREFIX: 100258,
        FIM_MIDDLE: 100259,
        FIM_SUFFIX: 100260,
        ENDOFPROMPT: 100276,
    }
    return {
        "name": "cl100k_base",
        "pat_str": r"""'(?i:[sdmt]|ll|ve|re)|[^\r\n\p{L}\p{N}]?+\p{L}++|\p{N}{1,3}+| ?[^\s\p{L}\p{N}]++[\r\n]*+|\s++$|\s*[\r\n]|\s+(?!\S)|\s""",
        "mergeable_ranks": mergeable_ranks,
        "special_tokens": special_tokens,
    }

def o200k_base():
    mergeable_ranks = load_tiktoken_bpe(
        "https://openaipublic.blob.core.windows.net/encodings/o200k_base.tiktoken",
        expected_hash="446a9538cb6c348e3516120d7c08b09f57c36495e2acfffe59a5bf8b0cfb1a2d",
    )
```

cl100k_base is the tokenizer used by the original GPT-4, GPT-4 Turbo, and GPT-3.5 Turbo

Example text: "_hello I'M reading 12345!\n" where, "\n" represents a new line / line break.

GPT-2: ["_", "hello", " I", "'", "M", " reading", " 12345", "!", "\n"]

New regex: ["_hello", " I", "'M", " reading", " ", "123", "45", "!\n"]

The new regex keeps "_hello", "'M", and "!\n" together, but separates the space before the number and splits the digits into groups of up to three.

- English contractions are now case-insensitive: both 'm and 'M are recognized
- Numbers are split into groups of 1–3 digits: 12345 becomes "123" and "45", instead of one chunk
- Spaces before numbers are separated: " 42" becomes " " and "42"
- Words can include a preceding symbol: "_hello" can stay together, whereas GPT-2 separates "_" and "hello"
  Previously, only a preceding space could attach to a group of letters.
- Punctuation can include following line breaks: "!\n" can stay together, whereas GPT-2 separates "!" and "\n"

https://github.com/openai/tiktoken/blob/main/tiktoken_ext/openai_public.py

GPT-4 Special Tokens

special_tokens = {

ENDOFTEXT: 100257, Marks the end of a document or text sequence.

FIM_PREFIX: 100258, Ex: “Here is the text **before the gap**: *Useful for*”

FIM_MIDDLE: 100259, Ex: “Now **generate the missing text**.”

FIM_SUFFIX: 100260, Ex: “Here is the text **after the gap**: *autocomplete.*”

ENDOFPROMPT: 100276, Marks the end of the supplied prompt, after which the model generates its completion.

}

Useful for code autocomplete.

fim_prefix fim_middle fim_suffix

Example:

- The model receives: **<|fim_prefix|>** def double(x): **<|fim_suffix|>** return result **<|fim_middle|>**
- And then generates the missing middle: “result = 2*x” (we ommited newline \n and spaces for readability)

Bavarian, Mohammad, et al. "Efficient training of language models to fill in the middle."

Special Tokens Enable Tool Use

Tiktokenizer

cl100k_base

```
<|endoftext|> vs endoftext
<|fim_prefix|> vs fim_prefix
<|fim_middle|> vs fim_middle
<|fim_suffix|> vs fim_suffix
<|endofprompt|> vs endofprompt
```

Token count
27

<|endoftext|> vs endoftext
<|fim_prefix|> vs fim_prefix
<|fim_middle|> vs fim_middle
<|fim_suffix|> vs fim_suffix
<|endofprompt|> vs endofprompt

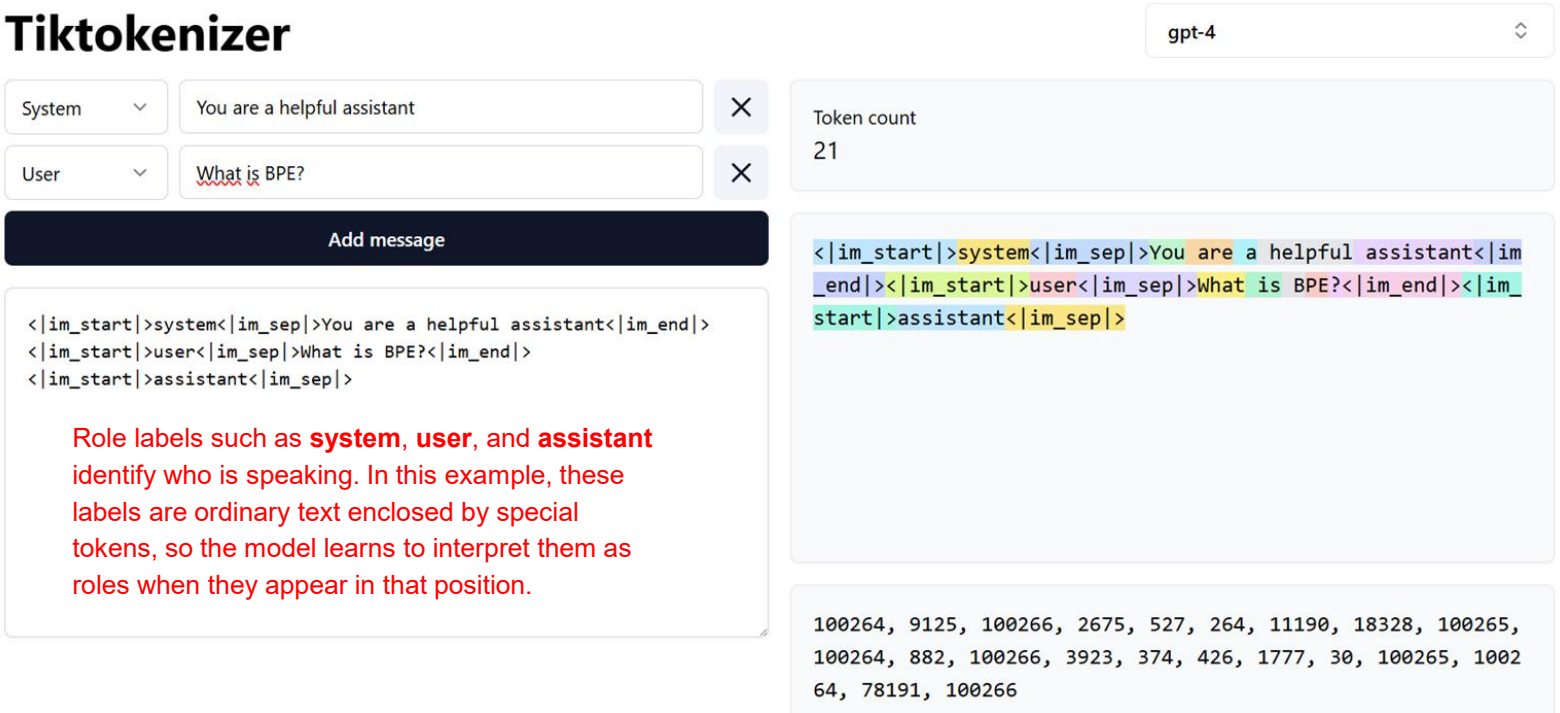
Note how the special tokens are a single block.

Special tokens are indivisible markers that structure messages and tool calls, e.g. `<message>` and `<call>` (illustrative names). Tool names, arguments, and content typically use **ordinary tokens**. The model learns the format during training, while external software executes tool calls.

100257, 6296, 842, 14450, 428, 198, 100258, 6296, 53187, 14
301, 198, 100259, 6296, 53187, 63680, 198, 100260, 6296, 53
187, 38251, 198, 100276, 6296, 842, 1073, 41681, 198

Search ***"special_tokens"*** → https://github.com/openai/tiktoken/blob/main/src/lib.rs

Structuring Conversations with Special Tokens



Special tokens mark the boundaries between messages and separate message headers from their content.

- **<|im_start|>** marks the beginning of a message.
- **<|im_sep|>** separates the message header from its content.
- **<|im_end|>** marks the end of a message.

The tiktoken Library

tiktoken

tiktoken is a fast BPE tokeniser for use with OpenAI's models.

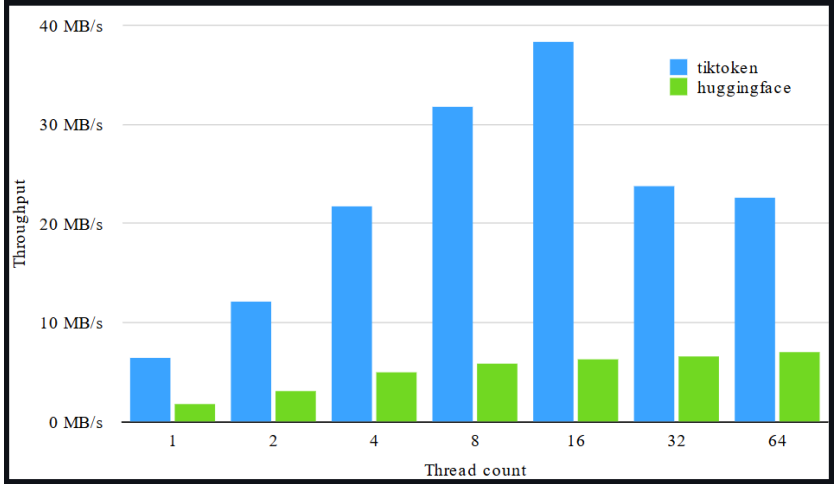
```
import tiktoken
enc = tiktoken.get_encoding("o200k_base")
assert enc.decode(enc.encode("hello world")) == "hello world"

# To get the tokeniser corresponding to a specific model in the OpenAI API:
enc = tiktoken.encoding_for_model("gpt-4o")
```

The open source version of `tiktoken` can be installed from PyPI:

```
pip install tiktoken
```

`tiktoken` is between 3-6x faster than a comparable open source tokeniser:



Performance measured on 1GB of text using the GPT-2 tokeniser, using `GPT2TokenizerFast` from `tokenizers==0.13.2`, `transformers==4.24.0` and `tiktoken==0.2.0`.

https://github.com/openai/tiktoken

Extending tiktoken

README MIT license

Extending tiktoken

You may wish to extend `tiktoken` to support new encodings. There are two ways to do this.

Create your `Encoding` object exactly the way you want and simply pass it around.

```
cl100k_base = tiktoken.get_encoding("cl100k_base")

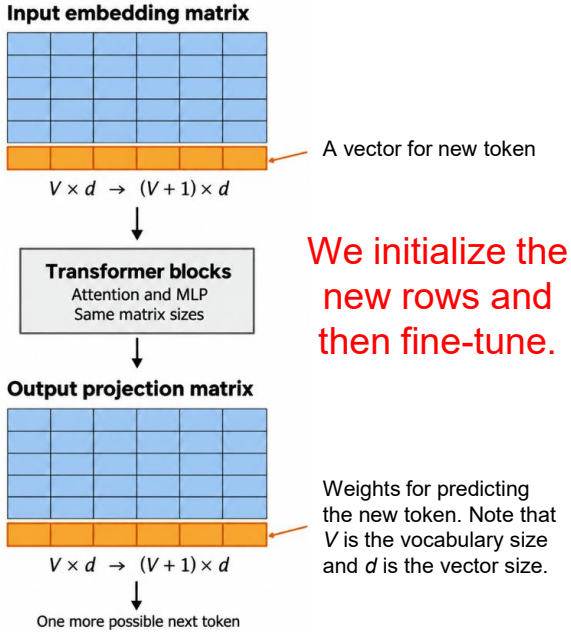
# In production, load the arguments directly instead of accessing private attributes
# See openai_public.py for examples of arguments for specific encodings
enc = tiktoken.Encoding(
    # If you're changing the set of special tokens, make sure to use a different name
    # It should be clear from the name what behaviour to expect.
    name="cl100k_im",
    pat_str=cl100k_base._pat_str,
    mergeable_ranks=cl100k_base._mergeable_ranks,
    special_tokens={
        **cl100k_base._special_tokens,
        "<|im_start|>": 100264,
        "<|im_end|>": 100265,
    }
)
```

You can add more special tokens with a new ID afterwards.

https://github.com/openai/tiktoken**#extending-tiktoken**

Adding New Tokens to a Pretrained Model

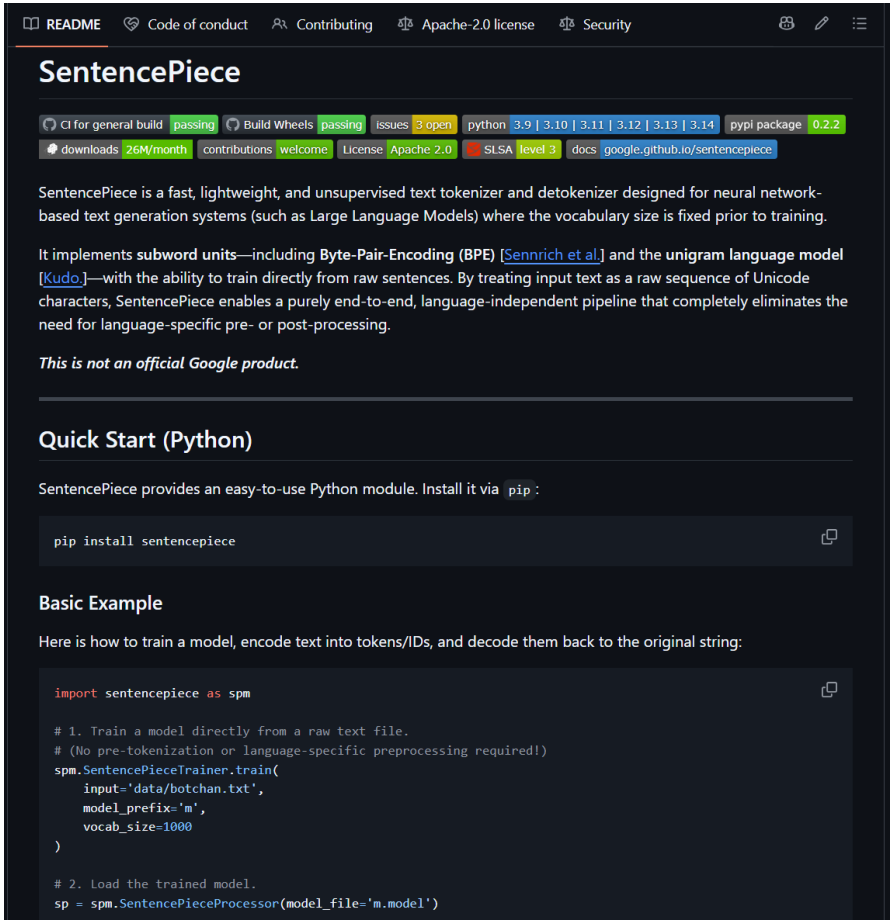
1. Add the new token to the tokenizer.
2. Expand the input embedding matrix and output vocabulary projection.
3. Initialize the new parameters.
4. Fine-tune on examples containing the new token.



We initialize the new rows and then fine-tune.

Adding special tokens can support adapting a **base model for chat** or introducing **tool-use formats** in chat and coding models. **Fine-tuning** teaches the model to interpret and generate these formats.

The SentencePiece Library



README | Code of conduct | Contributing | Apache-2.0 license | Security

SentencePiece

CI for general build passing | Build Wheels passing | issues 3 open | python 3.9 | 3.10 | 3.11 | 3.12 | 3.13 | 3.14 | pypi package 0.2.2
downloads 26M/month | contributions welcome | License Apache 2.0 | SLSA level 3 | docs google.github.io/sentencepiece

SentencePiece is a fast, lightweight, and unsupervised text tokenizer and detokenizer designed for neural network-based text generation systems (such as Large Language Models) where the vocabulary size is fixed prior to training.

It implements **subword units**—including **Byte-Pair-Encoding (BPE)** [Sennrich et al.] and the **unigram language model** [Kudo.]—with the ability to train directly from raw sentences. By treating input text as a raw sequence of Unicode characters, SentencePiece enables a purely end-to-end, language-independent pipeline that completely eliminates the need for language-specific pre- or post-processing.

This is not an official Google product.

Quick Start (Python)

SentencePiece provides an easy-to-use Python module. Install it via `pip`:

```
pip install sentencepiece
```

Basic Example

Here is how to train a model, encode text into tokens/IDs, and decode them back to the original string:

```
import sentencepiece as spm

# 1. Train a model directly from a raw text file.
# (No pre-tokenization or language-specific preprocessing required!)
spm.SentencePieceTrainer.train(
    input='data/botchan.txt',
    model_prefix='m',
    vocab_size=1000
)

# 2. Load the trained model.
sp = spm.SentencePieceProcessor(model_file='m.model')
```

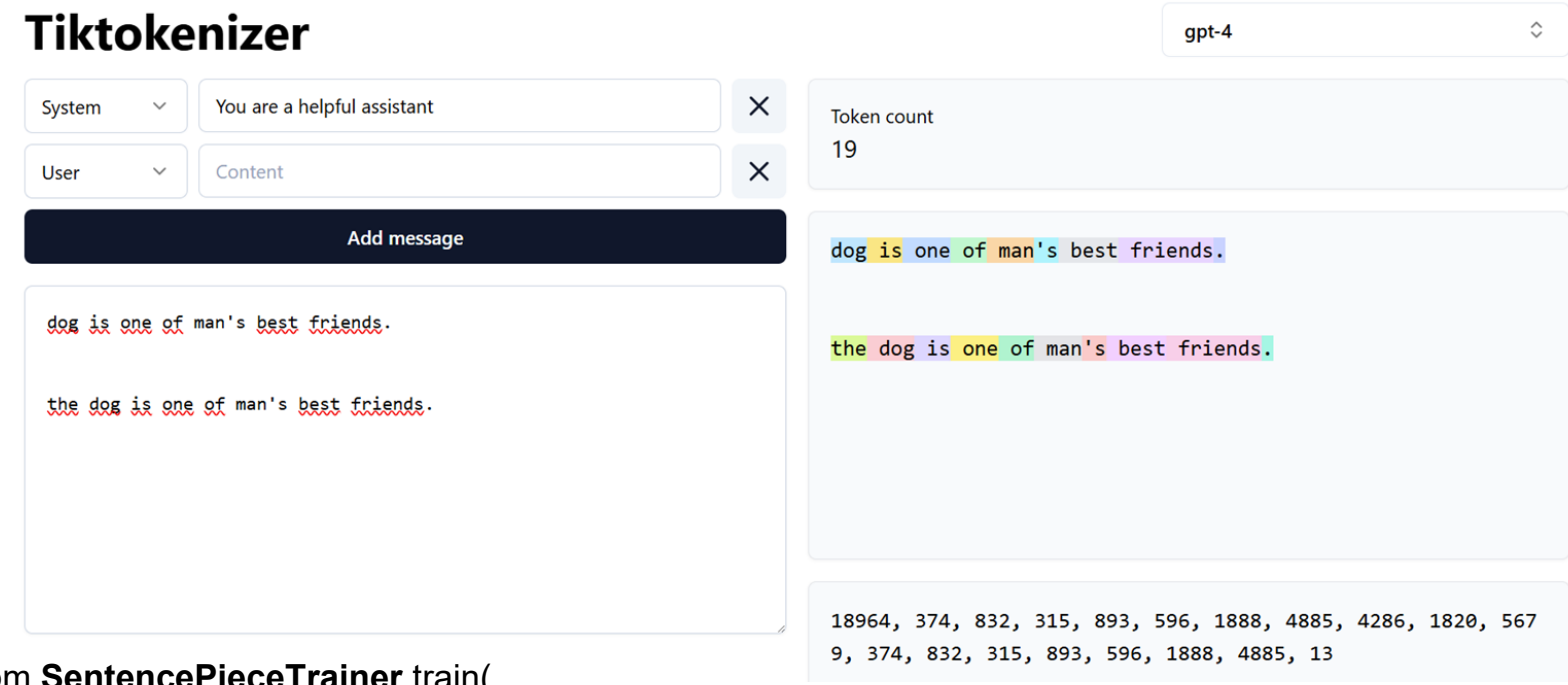
https://github.com/google/sentencepiece

- **SentencePiece** supports both **tokenizer training and inference**, whereas **tiktoken** focuses primarily on **inference with an already trained tokenizer**.
- **SentencePiece supports multiple algorithms**, including **BPE and Unigram**, whereas **tiktoken focuses on BPE**.
- **SentencePiece** is used by models such as **Llama 2**, **Mistral 7B**, and **Mixtral 8×7B**. (Llama 3 moved from SentencePiece to a tiktoken-based tokenizer)

What Is in a SentencePiece BPE Vocabulary?

1. **Special Tokens** → Reserved tokens with predefined functions, e.g. **<unk> for unknown tokens**, <s> for start of sentence, </s> for end of sentence, and optionally <pad> for padding.
2. **Byte Tokens** → Tokens representing individual bytes, used as a fallback for characters not covered by the normal vocabulary when **byte_fallback = True**
3. **Merge Tokens** → **Subword tokens** learned by repeatedly **merging** frequently occurring neighboring symbols during BPE training.
4. **Single-character tokens** → Tokens representing individual Unicode characters retained according to the specified character_coverage. If a codepoint is too rare to fall within the cumulative character coverage specified by character_coverage, it is not added to the vocabulary, e.g. **character_coverage = 0.995**

Whitespace Handling in SentencePiece




```
spm.SentencePieceTrainer.train(
        input="token_training_text.txt",
        model_prefix="model",
        vocab_size=8000,
        add_dummy_prefix=True
)
```

If **add_dummy_prefix** is **True**, a whitespace marker (_) will be added at the start of the input. This treats the first word like a word preceded by a space, reducing the need to learn separate token variants for the same word with and without a preceding space.

tiktoken vs SentencePiece

	tiktoken	SentencePiece
Basic representation	UTF-8 bytes	Unicode text / pieces
Tokenization methods	**BPE**	**BPE**, Unigram, character-level, word-level
Pre-tokenization	regex constraints	SentencePiece normalization / whitespace scheme
Training	primarily prebuilt encodings; educational trainer exists	full tokenizer-training toolkit
Whitespace	regex/BPE-dependent	explicit **“▁”** representation
Unknown characters	byte-based representation	**“<unk>”** or optional byte fallback

Takeaways

- **Tokens are the model's units of text.** The tokenizer converts text into a sequence of numerical IDs.
- **BPE builds tokens by merging frequent pairs.** Its training data shapes which patterns get compact representations.
- **Fewer tokens mean more room** in the context window. Different languages and formats use that space differently.
- **The tokenizer can split words in unexpected places.** Splits don't always follow syllables or familiar word parts. A token can also include the space before a word, so "hello" and " hello" may use different tokens depending on the tokenizer.
- **Special tokens provide structure.** The model learns the format during training, while external software executes tool calls.

References

Papers

- Gage (1994), *A New Algorithm for Data Compression* https://jacobfilipp.com/DrDobbs/articles/CUJ/1994/9402/gage/gage.htm [Slide 4, Speaker Notes Section]
- Sennrich, Haddow & Birch (2016). *Neural Machine Translation of Rare Words with Subword Units.* https://arxiv.org/abs/1508.07909 [Slide 4, Speaker Notes Section]
- Radford et al. (2019). *Language Models are Unsupervised Multitask Learners* (GPT-2). https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf [Slide 4, Speaker Notes Section]
- Bavarian et al. (2022). *Efficient Training of Language Models to Fill in the Middle.* https://arxiv.org/abs/2207.14255 [Slide 16, Speaker Notes Section]

Video

- Andrej Karpathy. *Let's build the GPT Tokenizer.* https://www.youtube.com/watch?v=zduSFxRajkE [Slide 23, Speaker Notes Section]

Code and libraries

- OpenAI GPT-2 encoder: https://github.com/openai/gpt-2/blob/master/src/encoder.py [Slide 14]
- tiktoken: https://github.com/openai/tiktoken [Slide 19]
- tiktoken encodings (cl100k_base regex): https://github.com/openai/tiktoken/blob/main/tiktoken_ext/openai_public.py [Slides 15, 16]
- tiktoken special tokens (lib.rs): https://github.com/openai/tiktoken/blob/main/src/lib.rs [Slide 17]
- Extending tiktoken: https://github.com/openai/tiktoken#extending-tiktoken [Slide 20]
- SentencePiece: https://github.com/google/sentencepiece [Slides 22, 23, 24, 25]
- Jupyter notebook: https://github.com/ducspe/UnderstandingTokenization/blob/main/BPE_Tokenizer_Notebook.ipynb [Slide 13]

Tools, courses and articles

- Tiktokenizer: https://tiktokenizer.vercel.app/ [Slides 7, 9, 10, 11, 17, 18, 24]
- Hugging Face LLM Course, BPE chapter: https://huggingface.co/learn/llm-course/en/chapter6/5 [Slide 5]
- Beren Millidge. *Integer tokenization is insane.* https://www.beren.io/2023-02-04-Integer-tokenization-is-insane/ [Slide 8]
- Nathan Reed. *A Programmer's Introduction to Unicode.* https://www.reedbeta.com/blog/programmers-intro-to-unicode/ [Slide 12]
- Wikipedia: UTF-8. https://en.wikipedia.org/wiki/UTF-8 [Slide 12]